

Phase 3 Vandermonde Encoder: DFT-node power sums and sort-encodings of the rep alignment table

Echinocoder / symcoder working notes

2026-05-26

Abstract

This document describes the mathematics behind `symcoder.encoders.phase3_vandermonde`, an alternative Phase 3 one-shot whole-table encoder for the symcoder pipeline. Like the simplicial-complex Phase 3 encoder it consumes the alignment table T of shape $(|G|, |\text{rep}|)$ and produces a permutation-invariant embedding; unlike the simplicial encoder it does so via polynomial projections (Vandermonde-power-sums) or via sorted-projections, both with a one-line correctness proof. The implementation uses complex roots-of-unity nodes (a DFT setup) for the C-infinity variant to achieve perfect numerical conditioning, and admits an optional dimensional “scale” parameter to fold out per-row unit mismatches.

Contents

1	Setting and notation	2
2	Vandermonde projection	2
3	Two ways to encode a multiset of n scalars	2
3.1	Power-sum encoding (C-infinity mode)	2
3.2	Sort encoding (C-zero mode)	3
3.3	Bridging projections: how many nodes are enough?	3
4	Numerical issues and the choice of nodes	3
4.1	Overflow with large-magnitude nodes	3
4.2	Underflow with bounded-magnitude real nodes	3
4.3	Unit-circle (DFT) nodes resolve both	4
5	From complex outputs to a real-valued vector	4
6	The optional scale parameter	4
6.1	The unit-mismatch problem	4
6.2	Dimensional homogenisation	4
7	Summary of guarantees and limitations	5
8	Code map	5

1 Setting and notation

We embed a multiset of n vectors in $\mathbb{R}^k$. Concretely the multiset is the column-set of the alignment table

$$T \in \mathbb{R}^{n \times k}, \quad T_{i,j} = \text{eval}(g_i \cdot \text{rep_atoms}[j], E),$$

where $g_1, \dots, g_n$ enumerate the group G (so $n = |G|$) and rep_atoms is the unpruned list of $|\text{rep}| = k$ canonical atoms over which Phase 3 encoders work. We treat T as a multiset of n rows of $\mathbb{R}^k$; the encoder must be invariant under permutation of rows.

The label-permutation group H acts on T by left-translation of g -index. An H -invariant encoding therefore inherits multiset invariance automatically.

2 Vandermonde projection

For each ‘‘Vandermonde node’’ $t_c \in \mathbb{C}$ (one per $c = 0, 1, \dots, m_{\text{proj}} - 1$), define the linear functional

$$L_c : \mathbb{R}^k \rightarrow \mathbb{C}, \quad L_c(x) = \sum_{j=1}^k t_c^{j-1} x_j.$$

Apply L_c to every row of T :

$$y_{c,i} = L_c(T_{i,\cdot}), \quad i = 1, \dots, n.$$

Then $y_{c,\cdot}$ is a multiset (or rather a sequence later treated as a multiset) of n values in $\mathbb{C}$, one per row of T .

3 Two ways to encode a multiset of n scalars

The encoder stores, for each c , an n -real representation of the multiset $\{y_{c,1}, \dots, y_{c,n}\}$. Two complementary choices are implemented.

3.1 Power-sum encoding (C-infinity mode)

The classical Newton-Girard identities relate the power sums $p_q = \sum_i y_i^q$ to the elementary symmetric polynomials e_q of $\{y_i\}$ via a triangular set of recursions. Knowing $p_1, p_2, \dots, p_n$ therefore determines $e_1, \dots, e_n$, which are the coefficients (up to sign) of the polynomial

$$\prod_{i=1}^n (X - y_i) = X^n - e_1 X^{n-1} + e_2 X^{n-2} - \dots \pm e_n,$$

whose roots are precisely $\{y_i\}$. Hence:

Lemma 1 (Univariate Newton-Girard). *The first n power sums $p_1, p_2, \dots, p_n$ uniquely determine the multiset of n scalars $\{y_1, \dots, y_n\}$.*

The C-infinity encoding for projection c is therefore

$$M_{c,p} = \sum_{i=1}^n y_{c,i}^p, \quad p = 1, 2, \dots, n.$$

Each $M_{c,p}$ is a polynomial in the entries of T , so the encoding is C^∞ everywhere.

3.2 Sort encoding (C-zero mode)

The same multiset can equivalently be encoded by its ascending sort:

$$S_{c,p} = \text{sort}(y_{c,1}, \dots, y_{c,n})_p, \quad p = 1, \dots, n.$$

Sorting is the canonical bijection between multisets of reals and ascending tuples of reals. $S_{c,p}$ is continuous and Lipschitz in the entries of T , but not differentiable at value-ties. It is “dimensionally homogeneous” with T (no powers are taken), which is why this mode is offered as an alternative to power sums.

3.3 Bridging projections: how many nodes are enough?

A multiset of n values in $\mathbb{C}$ determined per projection does not by itself recover the original multiset of n vectors in $\mathbb{R}^k$. We need enough projections to disambiguate the joint distribution. Expanding the p -th power sum:

$$M_{c,p} = \sum_i \left(\sum_j t_c^{j-1} T_{i,j} \right)^p = \sum_{(a_1, \dots, a_k): |a|=p} \binom{p}{a} t_c^{(j-1)a_j} \underbrace{\sum_i \prod_j T_{i,j}^{a_j}}_{\text{multi-symmetric power sum } P_a(T)}.$$

So $M_{c,p}$ is a polynomial in t_c of degree $(k-1)p$ whose coefficients are the multivariate (multi-symmetric) power sums $P_a(T)$ of total degree $|a| = p$.

Theorem 1. *If $t_0, \dots, t_{m-1}$ are m distinct complex numbers with $m \geq (k-1)n + 1$, the values $\{M_{c,p} : 0 \leq c < m, 1 \leq p \leq n\}$ determine all multi-symmetric power sums of total degree $\leq n$ of the row-multiset of T . Those in turn determine the row-multiset itself.*

Proof sketch. For each p , $M_{c,p}$ as a function of t is a polynomial of degree $(k-1)p \leq (k-1)n$ in t . Knowing it at $(k-1)n + 1$ distinct points determines it (Vandermonde non-singularity). Hence all $P_a(T)$ for $|a| \leq n$ are recovered. The multi-symmetric analogue of Newton-Girard (*multi-symmetric power sums up to total degree n separate multisets of n tuples*) then yields the multiset of rows. $\square$

The sort-encoding admits the same statement: per projection, $\{S_{c,1}, \dots, S_{c,n}\}$ is equivalent in information content to $\{M_{c,1}, \dots, M_{c,n}\}$ by Newton-Girard, and the Vandermonde / multi-symmetric argument is identical.

4 Numerical issues and the choice of nodes

Theorem 1 requires only that the t_c be distinct; their magnitudes are otherwise free. Two issues constrain the actual choice in float64 arithmetic.

4.1 Overflow with large-magnitude nodes

The naive choice $t_c = c + 1$ (distinct positive integers) gives $t_{m-1}^{k-1} \approx m^{k-1}$. For the parity-fixture plan ($m = 289$, $k = 25$) this is $289^{24} \approx 10^{58}$, which overflows float64 at the projection matrix stage before any sum is even taken.

4.2 Underflow with bounded-magnitude real nodes

Putting $|t_c| < 1$ (e.g. Chebyshev nodes in $(-1, 1)$) fixes overflow. But t_c^{k-1} then becomes very small for nodes near zero, so those projection-matrix entries silently underflow to zero. The Vandermonde matrix becomes ill-conditioned even when no element literally overflows or underflows.

4.3 Unit-circle (DFT) nodes resolve both

The choice

$$t_c = \exp\left(\frac{2\pi i c}{m_{\text{proj}}}\right), \quad c = 0, 1, \dots, m_{\text{proj}} - 1$$

gives the m_{proj} -th roots of unity. All are distinct (so Theorem 1 applies). All have modulus exactly 1, so $|t_c^{j-1}| = 1$ for every j : no overflow, no underflow. Furthermore the resulting Vandermonde matrix is (up to a unitary normalisation) the DFT matrix, which has condition number 1 exactly. This is the gold-standard numerical choice.

5 From complex outputs to a real-valued vector

With complex t_c the projections $y_{c,i}$ and power sums $M_{c,p}$ are complex. To package the encoding as a real vector we exploit conjugate symmetry.

Lemma 2. $M_p(t) = \sum_i (L_t(T_{i,\cdot}))^p$, viewed as a function of $t \in \mathbb{C}$, is a polynomial in t with real coefficients. Therefore $M_p(\bar{t}) = \overline{M_p(t)}$.

Proof. $L_t(T_{i,\cdot}) = \sum_j t^{j-1} T_{i,j}$ with real $T_{i,j}$, so the coefficient of t^d in $L_t(T_{i,\cdot})$ is real. Raising to the p -th power and summing over i preserves reality of coefficients. $\square$

Consequence. The lower-half roots of unity carry no additional information beyond the upper-half ones: M_p at $\bar{t}$ is determined by M_p at t . So we store only the upper-half (and real-axis) nodes. For m_{proj} odd there is exactly one real-axis node $t = +1$ at $c = 0$; for m_{proj} even there are two, $t = +1$ at $c = 0$ and $t = -1$ at $c = m_{\text{proj}}/2$. The encoding layout:

- $c = 0$ (node $t = +1$): emit $\text{Re}(M_{0,p})$ for $p = 1, \dots, n$. (n reals; Im is zero by the lemma.)
- For each $c = 1, \dots, \lfloor (m_{\text{proj}} - 1)/2 \rfloor$ (upper-half nodes): emit $\text{Re}(M_{c,p})$ and $\text{Im}(M_{c,p})$ interleaved, for $p = 1, \dots, n$. ($2n$ reals per such c .)
- If m_{proj} even, $c = m_{\text{proj}}/2$ (node $t = -1$): emit $\text{Re}(M_{c,p})$ for $p = 1, \dots, n$. (n reals.)

Total reals:

$$n + 2n \cdot \left\lfloor \frac{m_{\text{proj}} - 1}{2} \right\rfloor + [m_{\text{proj}} \text{ even}] \cdot n = n \cdot m_{\text{proj}}.$$

The same total size as if we had used m_{proj} real-valued nodes, but with perfect numerical conditioning.

6 The optional scale parameter

6.1 The unit-mismatch problem

In the C-infinity mode the encoding $M_{c,p}$ has the same physical units as y^p . But the columns of T themselves have different units when the rep atoms are of different mass dimension: mag columns are in units of length, dot columns in length-squared, ε_3 columns in length-cubed. Mixing these in the projection $L_c(T_{i,\cdot})$ and then raising to the p -th power gives output magnitudes that span $\sim |y|^1$ to $\sim |y|^n$ across p . For $n = 12$ that is twelve orders of magnitude of dynamic range baked into the encoding before anything else has happened.

6.2 Dimensional homogenisation

We define the *mass dimension* of an operation as the homogeneity degree of its eval_fn under uniform scaling of its vector arguments:

$$\text{eval_fn}(kv_1, \dots, kv_r) = k^{\text{mass.dim}} \text{eval_fn}(v_1, \dots, v_r).$$

Concretely `mag: 1`, `magSq: 2`, `dot: 2`, `$\varepsilon_3: 3$` . This degree is declared on the `Operation` object as the new `mass_dimension` field.

Given a user-supplied length scale $\lambda > 0$, the encoder first divides each column of T by λ raised to that column’s mass dimension:

$$T'_{i,j} = T_{i,j} / \lambda^{\text{mass_dim}(j)}.$$

The resulting T' is dimensionless and has entries of order unity when λ matches the natural scale of the event. Power sums of T' stay within a few orders of magnitude regardless of p .

Why λ must be external, not derived. A natural temptation is to set λ to (say) the largest absolute entry of T at encoding time. This is unsafe: the event values can be arbitrarily close to zero (e.g. the parity-fixture event with $p = (0, 0, 1)$, $q = (0, 0, -1)$ has many zero entries in T), so no data-derived λ is bounded away from zero. A bad λ amplifies rather than reduces dynamic range.

The scale is therefore a fixed encoder parameter and is recorded in the segment descriptor so that downstream consumers can undo it.

7 Summary of guarantees and limitations

- **Faithfulness.** The encoder is deterministically faithful (separates distinct row-multisets of T) by Theorem 1. No genericity assumption; no “measure-zero failures”.
- **Smoothness.** Cinf mode is C^∞ in T (polynomials). C0 mode is Lipschitz continuous but not differentiable at value-ties.
- **Numerical conditioning.** Cinf mode uses DFT nodes, giving an implicit Vandermonde matrix of condition number 1 and no over/underflow in the projection step.
- **Output size.** $m_{\text{proj}} \cdot n$ reals where $m_{\text{proj}} = (k - 1)n + 1$, so $\mathcal{O}(n^2k)$. Larger than the simplicial encoder’s $2nk + 1 - k = \mathcal{O}(nk)$ but with a one-line correctness proof and a smooth output (Cinf) or a dimensionally-homogeneous one (C0).
- **Scale parameter.** In Cinf mode an external length scale λ may be supplied to keep $M_{c,p}$ magnitudes bounded across p . Doesn’t affect faithfulness; only numerical conditioning.
- **What it does not do.** Does not address the unit mismatch *within* a single power p across different c . Does not provide an order- $\mathcal{O}(nk)$ alternative to the simplicial encoder; that remains an open problem.

8 Code map

File	Role
<code>symatom/atoms.py</code>	Adds the <code>mass_dimension</code> field to <code>Operation</code> .
<code>symcoder/operations/euclidean{2,3}.py</code>	Declares mass dimensions on the standard library operations.
<code>symcoder/encoders/phase3-common.py</code>	Shared helpers: <code>build_alignment_table</code> , <code>column_mass_dimensions</code> , <code>apply_scale</code> .
<code>symcoder/encoders/phase3_vandermonde.py</code>	This encoder. <code>Phase3VandermondeEncoder</code> + <code>Phase3VandermondeEncoderFactory</code> .
<code>symcoder/encoders/phase3_factory.py</code>	Stacking wrapper. <code>Phase3EncoderFactory</code> composes any number of Phase 3 sub-encoders.
<code>symcoder/encoders/phase3_simplicial.py</code>	The simplicial alternative. Same input (T), different output style.